

AI/ML Task 3 Project Report

Project Title: California House Price Prediction using Model Validation, Overfitting Control and Hyperparameter Tuning

Dataset: California Housing Dataset

Target Variable: Median House Value / HousePrice

Tools Used: Python, Jupyter Notebook, pandas, NumPy, matplotlib, scikit-learn, joblib

1. Objective

The objective of this project is to build a reliable machine learning regression model for predicting house prices using the California Housing Dataset. The main focus of Task 3 is not only accuracy, but model reliability. Therefore, the project includes train-test evaluation, overfitting analysis, cross-validation, hyperparameter tuning using GridSearchCV, and final optimized model selection.

2. Dataset Description

The California Housing Dataset contains numerical housing and location-related features. The input features include median income, house age, average rooms, average bedrooms, population, average occupancy, latitude, and longitude. The output variable is HousePrice, which represents median house value. The dataset is suitable for regression because the target is a continuous numeric value.

Column Type	Description
Input Features	Income, rooms, population, occupancy, latitude, longitude and related housing variables
Target	HousePrice / Median house value
Problem Type	Regression
Evaluation Metrics	MAE, RMSE, R2 Score

3. Methodology

First, the dataset is loaded using scikit-learn. The target column is separated from the input features. Feature scaling is applied using StandardScaler so that all input columns have comparable scale. The dataset is split into 80% training data and 20% testing data using train_test_split with random_state=42 for reproducibility.

Three baseline models are trained: Linear Regression, Ridge Regression, and Decision Tree Regressor. These models are evaluated using train and test RMSE and R2 Score. After baseline evaluation, 5-fold cross-validation is applied to get a more reliable performance estimate. Finally, Decision Tree hyperparameters are tuned using GridSearchCV.

4. Overfitting Analysis

Overfitting happens when a model performs very well on training data but performs poorly on new testing data. In this project, overfitting is detected by comparing train RMSE and test RMSE. If train RMSE is very low and test RMSE is much higher, the model is likely overfitting.

Decision Tree models are more likely to overfit because they can keep splitting the data until the tree memorizes training examples. To control this, the project tunes max_depth, min_samples_split, and min_samples_leaf. These parameters restrict tree complexity and improve generalization.

5. Cross-Validation

A single train-test split can sometimes give unstable results. Cross-validation solves this problem by splitting the dataset into multiple folds. In this project, 5-fold cross-validation is used. The model trains and validates five times on different parts of the data, and the average RMSE is used as the cross-validation score. This gives a more trusted estimate of model performance.

6. Hyperparameter Tuning

Hyperparameters are settings chosen before model training. GridSearchCV automatically tries different combinations of hyperparameter values and selects the best combination based on cross-validation RMSE. For Decision Tree, the tuned parameters are max_depth, min_samples_split, and min_samples_leaf.

Hyperparameter	Purpose
max_depth	Controls maximum tree depth and reduces memorization
min_samples_split	Controls minimum samples needed to split a node
min_samples_leaf	Controls minimum samples allowed in a final leaf

7. Model Comparison

The models are compared using RMSE and R2 Score. RMSE measures prediction error, so a lower RMSE is better. R2 Score measures how much variance in the target variable is explained by the model, so a higher R2 Score is better. The final comparison table in the notebook shows baseline models and the tuned Decision Tree together.

The final model should be selected based on: lowest test RMSE, highest test R2 Score, smaller overfitting gap, and stable cross-validation performance. The tuned Decision Tree is expected to reduce overfitting compared to the untuned Decision Tree because its complexity is controlled by the selected hyperparameters.

8. Final Model Justification

The final model is selected scientifically by comparing performance on test data and cross-validation. Cross-validation results are trusted because they evaluate the model on multiple data splits instead of only one train-test split. Hyperparameter tuning improves model behavior by preventing excessive complexity and reducing overfitting. A model with good generalization is more useful in real-world machine learning projects than a model that only performs well on training data.

9. Conclusion

This project demonstrates an industry-aligned machine learning workflow. The project starts with baseline model training and then moves to professional validation methods. Overfitting is detected using train versus test performance, cross-validation provides a stable estimate, and GridSearchCV improves the Decision Tree model through systematic tuning. The final selected model is more reliable for unseen data and can be saved using joblib for future use.

10. Deliverables Included

1. Complete Jupyter Notebook: AI_ML_Task3_Model_Validation_Tuning_Complete_Project.ipynb
2. PDF Report: AI_ML_Task3_Project_Report.pdf
3. Model saving code for best_house_price_model.pkl and scaler.pkl
4. Charts for model comparison and overfitting analysis
5. Final conclusion and model selection justification